Generative AI and Its Applications

Unit 3

(UE23CS342BA4)

Dense Retrieval and Hybrid Retrieval

Dr. Pooja Agarwal

Dept. of CSE, PES University, Bangalore

Acknowledgement

The slides are prepared from various resources from the Universities from abroad and India. Also, some material is taken from reliable resources from internet throughout this course.

The Vocabulary Mismatch Problem

The Fundamental Challenge in Information Retrieval!

Query: "How to fix a car that won't start?"

Traditional Sparse Retrieval (BM25) looks for:
"fix" AND "car" AND "won't" AND "start"

But relevant documents might say:
"automobile engine troubleshooting guide" No match!
"vehicle ignition problems solutions" No match!
"jump-start dead battery procedure" No match!
"starter motor replacement steps" No match!

THEREFORE Result: ZERO relevant documents retrieved!

Sparse retrieval fails because keywords differ.

Analogy:
Sparse retrieval is like a librarian who only looks for exact book titles. If you ask for "automobile repair," they ignore books about "car maintenance" because the words don't match exactly.

Dense Retrieval!

Dense Retrieval is a technique that represents queries and documents as dense vectors (embeddings) in a continuous semantic space, where similarity is measured by vector distance rather than keyword matching.

Text → Neural Encoder → Vector

"cat" → [0.23, -0.45, 0.67, 0.12, ..., 0.34] (768 dimensions)
"feline" → [0.21, -0.44, 0.65, 0.15, ..., 0.36] (similar vectors!)

Vectors with similar meaning are close together.

What is Dense Retrieval?

Dense Retrieval!

Dense Retrieval is a technique that represents queries and documents as dense vectors (embeddings) in a continuous semantic space, where similarity is measured by vector distance rather than keyword matching.

Step-by-Step Steps:

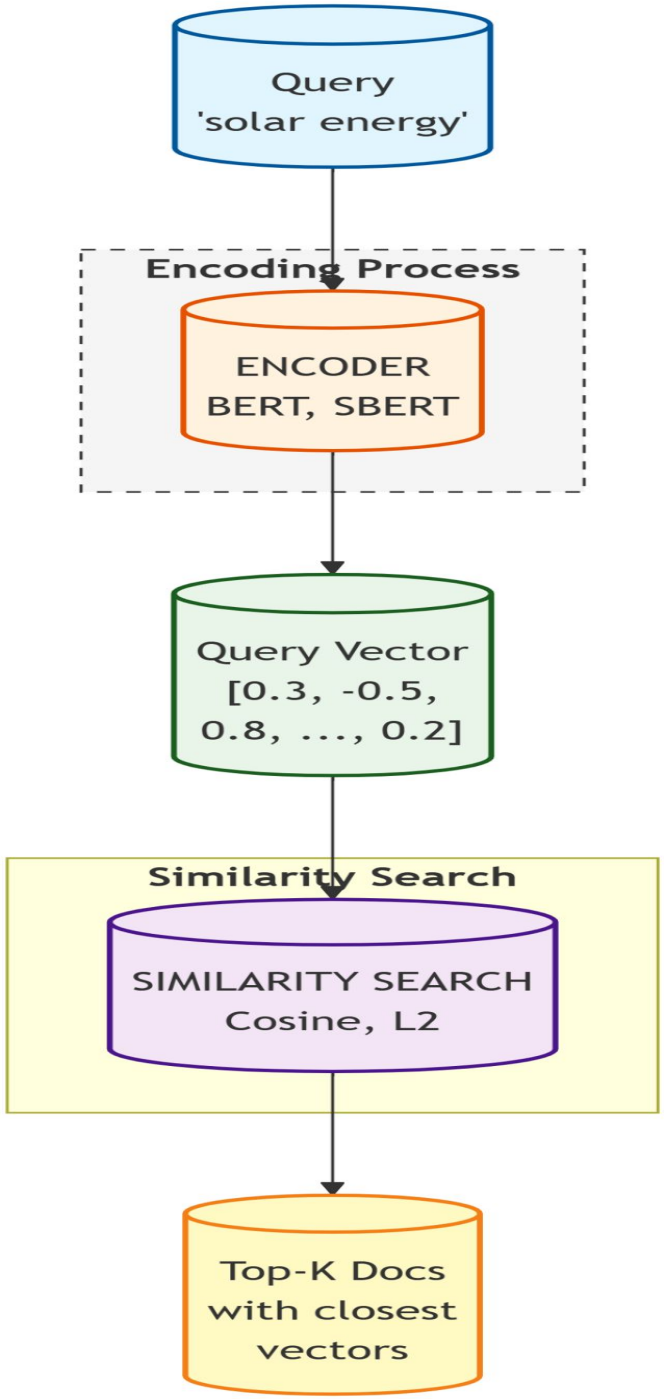
1. Encode documents into embeddings
2. Store embeddings in vector database
3. Encode query
4. Compute similarity
5. Retrieve top-K documents

Architecture:

Query → Encoder → Vector

Docs → Encoder → Vectors

Similarity → Ranking



1. **Encode query** → dense vector
2. **Encode all documents** → dense vectors (pre-computed)
3. **Compare** query vector to all document vectors
4. **Return** documents with highest similarity scores

Dense V/S Sparse Overview!

Aspect	Sparse Retrieval (BM25)	Dense Retrieval (Embeddings)
Representation	Bag-of-Words / Term Frequency	Semantic Vector Embeddings
Dimensions	Vocabulary size (50K – 1M)	Fixed small vector (384 – 768)
Vector Structure	Very sparse (≈99.9% zeros)	Dense (almost all values non-zero)
Matching Method	Exact keyword matching	Semantic similarity matching
Synonym Handling	Misses synonyms	Captures synonyms
Polysemy (multiple meanings)	Often confused	Context-aware understanding
Interpretability	High (terms visible)	Low (latent features)
Training Required	No training needed	Requires trained model
Speed	Very fast	Slower (but optimized with ANN search)

Query: "bank"
Sparse retrieves:
- Documents with "bank" (both river bank AND financial bank)

Dense with context:
- "I went to the bank to deposit money" → Financial context
- "The river bank eroded" → Geographic context

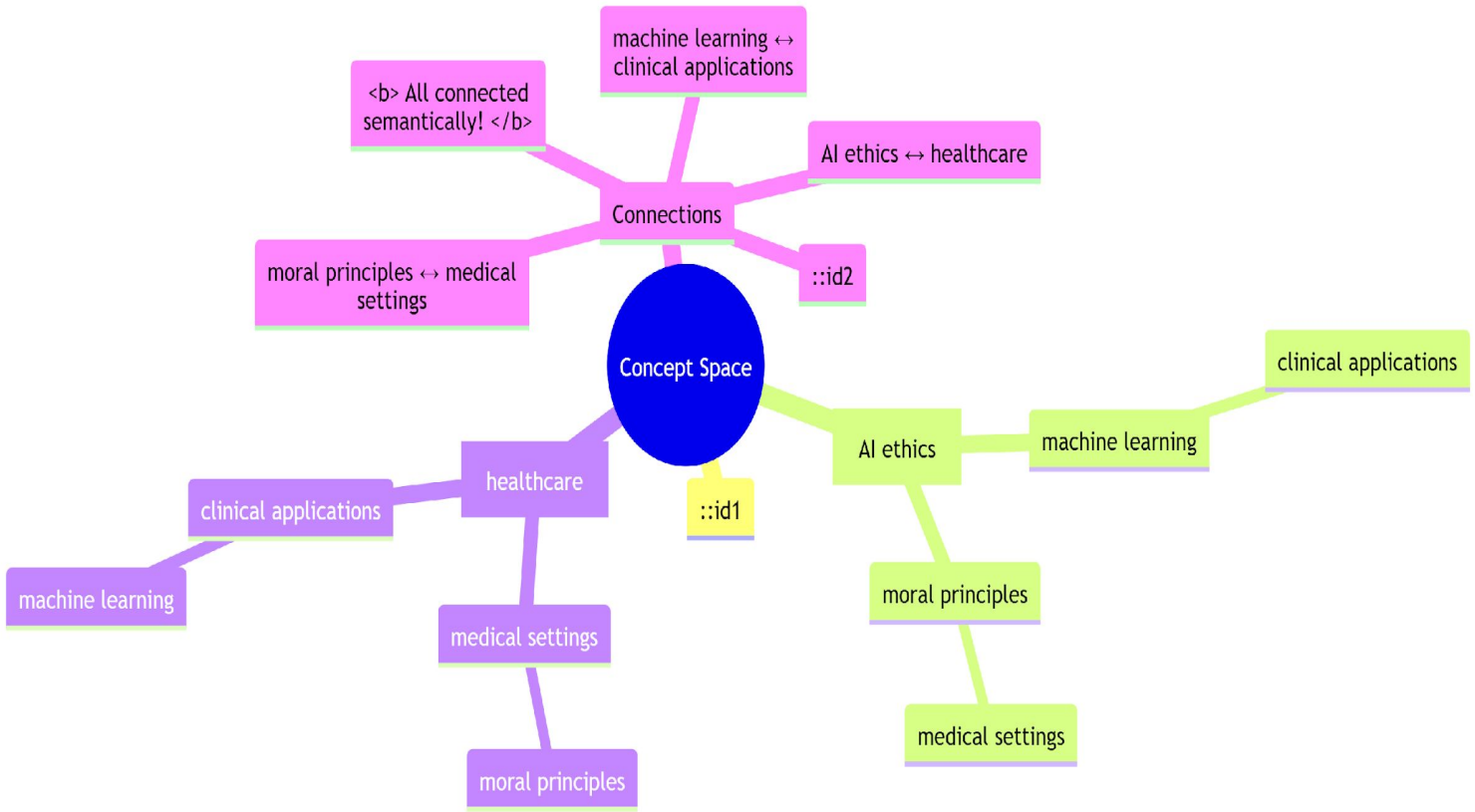
GenerativeAI and Its Applications

Why Dense Retrieval? The Semantic Gap

Query: "AI ethics guidelines for healthcare"

Documents that should match but don't contain these exact words:
1. "Artificial intelligence moral principles in medical settings"
2. "Machine learning ethical frameworks for clinical applications"
3. "Responsible AI deployment in hospitals: best practices"

BM25 Score: 0 (no common terms!)
Dense Score: High (similar meaning!)



Why Dense Retrieval? The Semantic Gap

Words with **similar meaning form clusters**.

Advantages:

- Understands synonyms
- Understands context
- Supports semantic search

EXAMPLE:

car — automobile — vehicle
dog — puppy — canine
doctor — physician

GenerativeAI and Its Applications

Similarity Measurement

document = "The cat sat on the mat"

↓

Embedding Model (BERT/SBERT)

↓

Vector d = [0.23, -0.56, 0.78, 0.12, ..., -0.34] $\in \mathbb{R}^{768}$

Similarity Metrics:

1. **Cosine Similarity** (Most common)

$$\cos(\theta) = \frac{q \cdot d}{\|q\| \, \|d\|} = \frac{\Sigma(q_i \times d_i)}{\sqrt{(\Sigma q_i^2)} \times \sqrt{(\Sigma d_i^2)}}$$

Range: [-1, 1] ⟶ Higher = More similar

Value	Meaning
1	identical
0.8	highly similar
0	unrelated

2.Euclidean Distance (L2)

$$L2(q,d) = \sqrt{[\Sigma(q_i - d_i)^2]}$$

Range: [0, ∞) → Lower = More similar

3.Dot Product (when vectors are normalized)

$$dot(q,d) = \Sigma(q_i \times d_i)$$

Range: [-1, 1] → Higher = More similar

LETS SOLVE:

Dense Retrieval Numerical Example

2D vectors (simplified for demonstration)

Query: "solar power"

- Document 1: "renewable energy"
- Document 2: "financial investment"

Step 1: Get Embeddings

GenerativeAI and Its Applications

Similarity Measurement

cosine similarity formula:

$$\cos(\theta) = (A \cdot B) / (||A|| \; ||B||)$$

Where

- $A \cdot B$ = dot product
- $||A||$ = magnitude of vector A

Step 1: Given Embeddings

Text	Vector
Query q = "solar power"	**[0.8 , 0.6]**
Doc1 d_1 = "renewable energy"	**[0.9 , 0.4]**
Doc2 d_2 = "financial investment"	**[0.1 , 0.2]**

Step 2: Compute Similarity with Document 1

Dot Product

$$q \cdot d_1 = (0.8 \times 0.9) + (0.6 \times 0.4)$$
$$= 0.72 + 0.24 = 0.96$$

Vector Magnitudes

$$\| q \| = \sqrt{0.8^2 + 0.6^2}$$
$$= \sqrt{0.64 + 0.36} = \sqrt{1.00} = 1.00$$
$$\| d_1 \| = \sqrt{0.9^2 + 0.4^2}$$
$$= \sqrt{0.81 + 0.16} = \sqrt{0.97} = 0.985$$

Cosine Similarity

$$cos(\theta_1) = \frac{0.96}{1.00 \times 0.985}$$
$$cos(\theta_1) = 0.975$$

=High similarity

Step 3: Compute Similarity with Document 2

Dot Product

$$q \cdot d_2 = (0.8 \times 0.1) + (0.6 \times 0.2)$$
$$= 0.08 + 0.12 = 0.20$$

Magnitude

$$|| \, d_2 \, || = \sqrt{0.1^2 + 0.2^2}$$
$$= \sqrt{0.01 + 0.04}$$
$$= \sqrt{0.05} = 0.224$$

Cosine Similarity

$$cos(\theta_2) = \frac{0.20}{1.00 \times 0.224}$$
$$cos(\theta_2) = 0.893$$

=Less similarity

GenerativeAI and Its Applications

Similarity Measurement

Document	Cosine Similarity	Relevance
Doc1 – renewable energy	**0.975**	Most Relevant
Doc2 – financial investment	**0.893**	Less Relevant

Doc1 (0.975) > Doc2 (0.893)

Therefore **"renewable energy"** is **more relevant to the query "solar power"**.

GenerativeAI and Its Applications

Training Dense Retrievers

How Do We Get Good Embeddings?

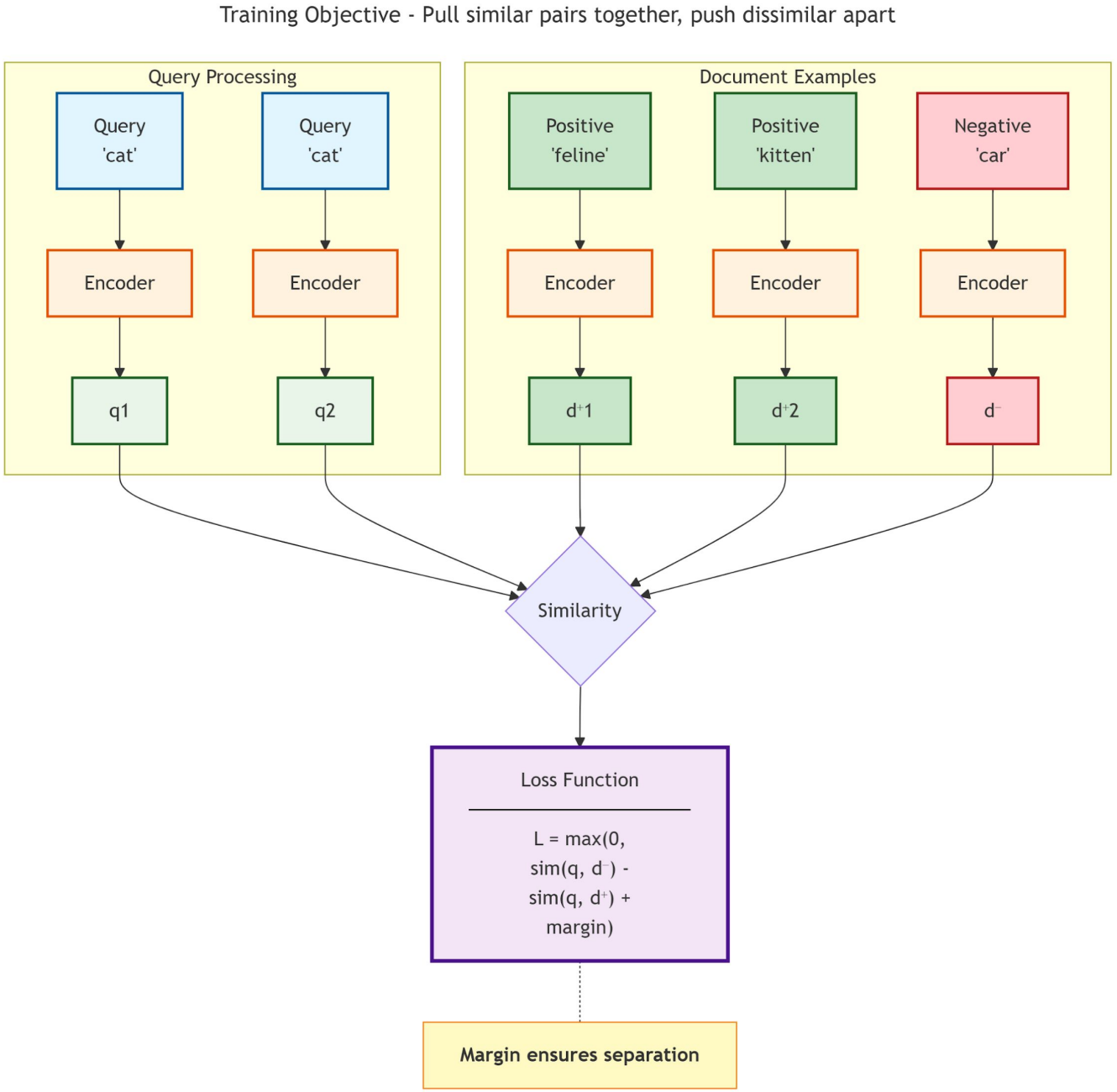
Triplet / Margin Ranking Loss (Used in Dense Retrieval Training)

Loss = max(0, cos(q,d⁻) - cos(q,d⁺) + margin)

This loss function is used to **train dense retrieval models like SBERT** so that:

- Relevant documents are **closer to the query**

Symbol	Meaning
q	Query embedding
d^+	Positive document (relevant)
d^-	Negative document (irrelevant)
cos(q,d)	Cosine similarity between query and document
margin	Minimum gap required between positive and negative similarity



How Do We Get Good Embeddings?

Example Calculation (Classroom Example)

Assume:

Value	Number
$\cos(q, d^+)$	0.85
$\cos(q, d^-)$	0.60
margin	0.20

Step 1: Substitute

```
Loss = max(0, 0.60 - 0.85 + 0.20)
```

Step 2: Solve

```
Loss = max(0, -0.05)
Loss = 0
```

Good training example

The model already satisfies the margin.

Value	Number
$\cos(q, d^+)$	0.70
$\cos(q, d^-)$	0.65
margin	0.20

```
Loss = max(0, 0.65 - 0.70 + 0.20)
Loss = max(0, 0.15)
Loss = 0.15
```

Bad example

The model must **adjust embeddings** so that:

$\cos(q,d^+)$ increases

$\cos(q,d^-)$ decreases

Dense Retrieval Similarity

Problem

A query and two documents are represented by **2-dimensional embeddings**.

Text	Vector
Query **q**	[0.6 , 0.8]
Document $\mathbf{d_1}$	[0.7 , 0.7]
Document $\mathbf{d_2}$	[0.2 , 0.1]

Task

1. Compute the **cosine similarity** between **q and** $\mathbf{d_1}$
2. Compute the **cosine similarity** between **q and** $\mathbf{d_2}$
3. Which document will the **dense retriever rank higher**?

Dense Retrieval Similarity Problem

Dense Retrieval Similarity

Cosine Similarity Formula

cos(θ) = (A · B) / (||A|| ||B||)

Step 1: Compute Dot Products

For **Document 1**

$$q \cdot d_1 = (0.6 \times 0.7) + (0.8 \times 0.7)$$
$$= 0.42 + 0.56 = 0.98$$

For **Document 2**

$$q \cdot d_2 = (0.6 \times 0.2) + (0.8 \times 0.1)$$
$$= 0.12 + 0.08 = 0.20$$

Step 2: Compute Vector Magnitudes

Query

$$\| q \| = \sqrt{0.6^2 + 0.8^2}$$
$$= \sqrt{0.36 + 0.64} = 1$$

Document 1

$$\| d_1 \| = \sqrt{0.7^2 + 0.7^2}$$
$$= \sqrt{0.49 + 0.49} = \sqrt{0.98} = 0.99$$

Document 2

$$\| d_2 \| = \sqrt{0.2^2 + 0.1^2}$$
$$= \sqrt{0.04 + 0.01} = \sqrt{0.05} = 0.224$$

Step 3: Cosine Similarities

Document 1

$$cos(q, d_1) = 0.98/(1 \times 0.99)$$
$$cos(q, d_1) \approx 0.99$$

Document 2

$$cos(q, d_2) = 0.20/(1 \times 0.224)$$
$$cos(q, d_2) \approx 0.89$$

Document $\mathbf{d_1}$ is more semantically similar to the query.

Dense Retrieval Similarity

Triplet Loss Problem

Value	Number
$\cos(q,d^{+})$	0.82
$\cos(q,d^{-})$	0.71
margin	0.2

Compute the **training loss**.

Dense Retrieval Similarity

Triplet Loss Problem

Loss=max(0,0.71−0.82+0.2)

$$Loss = max(0, 0.09)$$

$$Loss = 0.09$$

Model must learn more.

SBERT (SENTENCE-BERT) ?

Sentence-BERT (SBERT) is a modification of the BERT architecture that uses siamese network structures to derive semantically meaningful sentence embeddings that can be compared using cosine similarity.

The Problem with BERT:

BERT was designed for pair classification:
Input: [CLS] sentence A [SEP] sentence B [SEP] → similarity score

Problem: To find most similar sentence among 10,000, you'd need:
10,000 forward passes through BERT → IMPOSSIBLY SLOW!

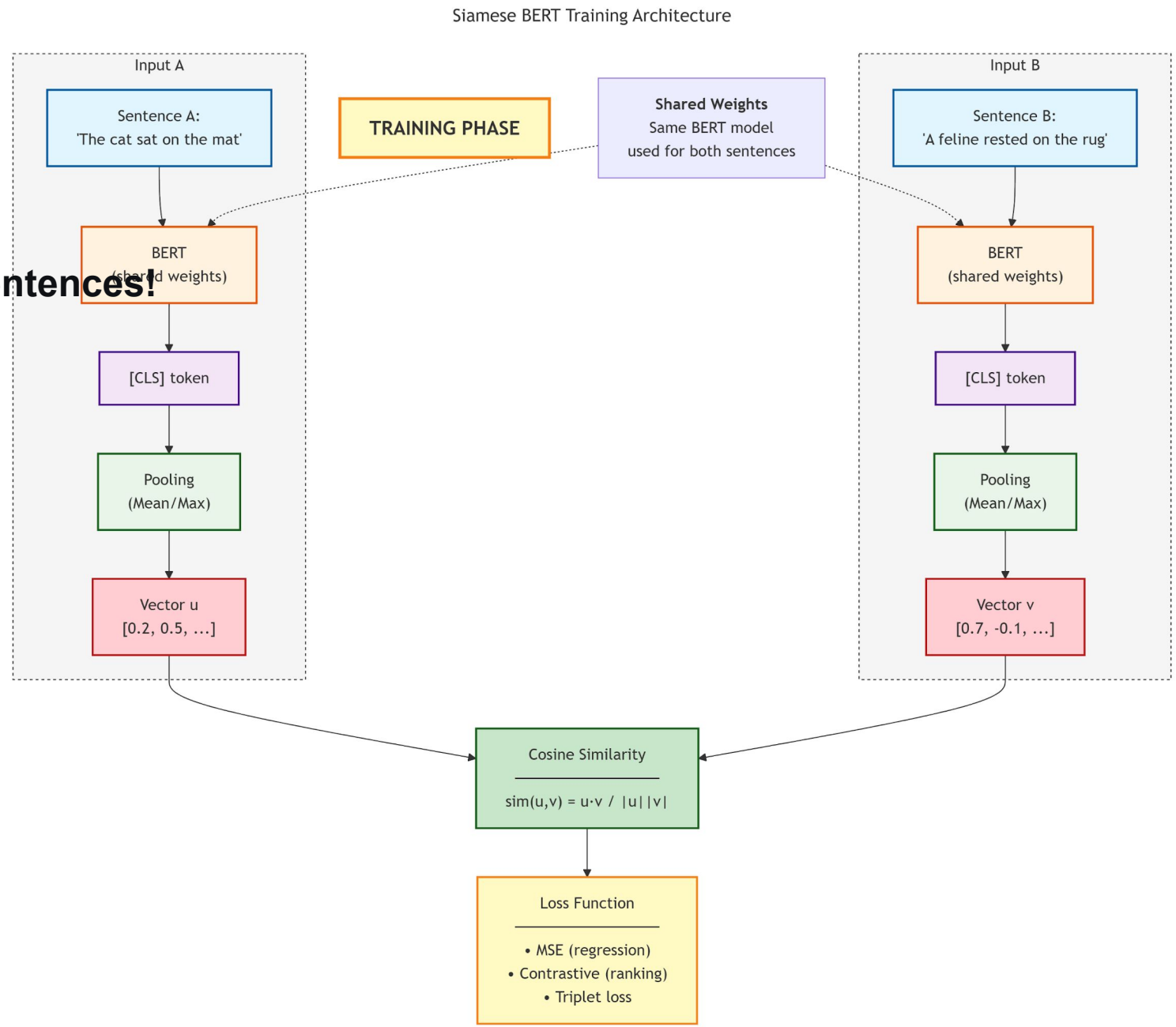
SBERT Solution: Pre-compute embeddings for all sentences once:
s1 → [0.2, -0.5, 0.7, ...]
s2 → [0.3, -0.4, 0.6, ...]
s3 → [-0.1, 0.8, -0.2, ...]
...
Then just compare vectors with cosine similarity!

GenerativeAI and Its Applications

SBERT (SENTENCE-BERT)

SBERT (SENTENCE-BERT) ?

Key Feature: Same BERT weights for both sentences!



SBERT Pooling Strategies?

How do we get a single vector from BERT's output?

BERT produces **one embedding per token**.

Example sentence:

`"The cat sat on the mat"`

BERT output:

`[token1_vec, token2_vec, token3_vec, ..., tokenN_vec]`

Each token vector has **768 dimensions**.

Need to be achieved: Convert these **N vectors** → **1 sentence vector**

Pooling Method	Idea	Formula	Notes
CLS Token Pooling	Use the special `[CLS]` token embedding	Sentence = CLS vector	Simple but sometimes unstable
Mean Pooling	Average all token embeddings	Mean(token vectors)	Most common in SBERT
Max Pooling	Take maximum value across tokens	Max(token vectors)	Captures strongest signals

SBERT Pooling Strategies?

Mean Pooling Formula

SentenceEmbedding = (token1 + token2 + ... + tokenN) / N

This creates a **single vector representing the entire sentence**.

Example

Sentence:

`"Machine learning is powerful"`

Token embeddings (simplified 3-dimensional):

Token	Vector
Machine	[0.4, 0.6, 0.2]
learning	[0.5, 0.7, 0.3]
powerful	[0.6, 0.8, 0.4]

Mean Pooling

```
Sentence vector =([0.4,0.6,0.2] + [0.5,0.7,0.3] + [0.6,0.8,0.4]) / 3
= [1.5 , 2.1 , 0.9] / 3
= [0.5 , 0.7 , 0.3]
```

Final **sentence embedding**

`[0.5 , 0.7 , 0.3]`

Method	Performance	Usage
CLS Pooling	Good	Used in original BERT
Mean Pooling	Best	Default in SBERT
Max Pooling	Moderate	Used in some NLP tasks

SBERT Training Objectives?

SBERT learns sentence embeddings by training on **sentence pairs or triplets**.
There are **three major training objectives**.

1.) Classification Objective: Predict whether two sentences are similar or not

Example:
Sentence A:
"I love programming"
Sentence B:
"Coding is my passion"
Label = **1 (similar)**
Model Input Representation:

```
[u ; v ; |u − v|]
```

where

- **u** = embedding of sentence A
- **v** = embedding of sentence B
- **|u−v|** = absolute difference

Loss Function:

```
Loss = CrossEntropy( softmax(W · [u ; v ; |u − v|]), label )
```

Explanation for students:
The model **classifies sentence pairs into categories** such as:

Label	Meaning
0	Not similar
1	Similar

SBERT Training Objectives?

SBERT learns sentence embeddings by training on **sentence pairs or triplets**.
There are **three major training objectives**.

2. Regression Objective

Goal: **Predict a similarity score between two sentences**

Example:

Sentence A: "The weather is nice"

Sentence B: "It's sunny today"

Target Similarity Score:0.8

Model computes **cosine similarity between embeddings**.

Cosine Similarity:

$\cos(\theta) = (u \cdot v) / (||u||\ ||v||)$

Loss Function:

```
Loss = MSE( cosine_sim(u,v), target )
```

Term	Meaning
MSE	Mean Squared Error
target	Ground truth similarity

Pair	Target
Sentence A vs B	0.8

The model learns to **predict similarity scores directly**.

SBERT Training Objectives?

3. Triplet Objective (Most Important for Retrieval)

Goal:

Learn embeddings so that

Anchor is closer to Positive than Negative

Example:

Anchor: "Machine learning is fascinating"

Positive: "AI and deep learning are interesting"

Negative: "I need to buy groceries"

Triplet Loss:

```
Loss = max(0, cos(anchor,negative) - cos(anchor,positive) + margin)
```

Margin usually:

```
0.1 - 0.3
```

Interpretation:

Relationship	Desired
cos(anchor,positive)	HIGH
cos(anchor,negative)	LOW

Semantic Space

```
        Positive
          ●
         /
        /
Anchor ●

        \
         \
          ● Negative
```

distance(anchor,positive) < distance(anchor,negative)

SBERT Training Objectives?

3. Triplet Objective (Most Important for Retrieval)

Goal:

Learn embeddings so that

Anchor is closer to Positive than Negative

Example:

Anchor: "Machine learning is fascinating"

Positive: "AI and deep learning are interesting"

Negative: "I need to buy groceries"

Triplet Loss:

```
Loss = max(0, cos(anchor,negative) - cos(anchor,positive) + margin)
```

Margin usually:

```
0.1 - 0.3
```

Interpretation:

Relationship	Desired
cos(anchor,positive)	HIGH
cos(anchor,negative)	LOW

Semantic Space

```
        Positive
          ●
         /
        /
Anchor ●

        \
         \
          ● Negative
```

distance(anchor,positive) < distance(anchor,negative)

SBERT Training Problem!

Training Data

Anchor (a): "deep learning" → [0.9, 0.8]
Positive (p): "neural networks" → [0.8, 0.7]
Negative (n): "grocery shopping" → [0.1, 0.2]
margin = 0.3

Step 1: Cosine Similarity

Calculate Norms:

$$\|a\| = \sqrt{(0.9^2 + 0.8^2)} = \sqrt{(0.81 + 0.64)} = \sqrt{1.45} = 1.204$$

$$\|p\| = \sqrt{(0.8^2 + 0.7^2)} = \sqrt{(0.64 + 0.49)} = \sqrt{1.13} = 1.063$$

$$\|n\| = \sqrt{(0.1^2 + 0.2^2)} = \sqrt{(0.01 + 0.04)} = \sqrt{0.05} = 0.224$$

Calculate: cos(a,p)

$$\cos(a,p) = (0.9\times0.8 + 0.8\times0.7) / (1.204 \times 1.063) = (0.72 + 0.56) / 1.279 = 1.28 / 1.279 \approx 0.999 \approx 1.00$$

Calculate: cos(a,n)

$$\cos(a,n) = (0.9\times0.1 + 0.8\times0.2) / (1.204 \times 0.224) = (0.09 + 0.16) / 0.269 = 0.25 / 0.269 \approx 0.93$$

SBERT Training Problem!

Training Data

Anchor (a): "deep learning" → [0.9, 0.8]
Positive (p): "neural networks" → [0.8, 0.7]
Negative (n): "grocery shopping" → [0.1, 0.2]
margin = 0.3

Step 2: Triplet Loss

Triplet loss formula:
Loss = max(0, cos(a,n) - cos(a,p) + margin)
Substitute values:
Loss = max(0, 0.93 - 1.00 + 0.3)
= max(0, 0.23)
= 0.23
Since **loss > 0**, the model must update weights.

Step 3: Backpropagation Intuition

The optimizer updates SBERT so that:
cos(a,p) → increases
cos(a,n) → decreases
Graphically:
Before training
a ---- p (close)
a ---- n (too close)

After training
a ---- p (very close)
a ---------- n (far away)

Triplet loss trains SBERT so that semantically similar sentences have **higher cosine similarity** than unrelated sentences by at least a **margin**.

SBERT Inference - Finding Similar Sentences!

Problem: Find sentences similar to query in 1M document corpus

Step 1: Pre-compute Document Embeddings (One-time cost)

Document 1: "Quantum computing uses qubits" → [0.23, -0.45, 0.67, ...]

Document 2: "Machine learning algorithms" → [-0.12, 0.78, -0.34, ...]

Document 3: "Climate change effects" → [0.56, 0.23, -0.89, ...]

...

Document 1M: "..." → [...]

Time: ~1 second per 1000 docs = ~1000 seconds total

SBERT Inference - Finding Similar Sentences!

Step 2: Encode Query (At search time)

Query: "What is quantum computing?" → q = [0.25, -0.40, 0.70, ...]

Time: ~0.01 seconds

Step 3: Similarity Search

For each doc i:

score_i = cosine_similarity(q, doc_i)

Time with flat index: O(n × d) = 1M × 768 operations ≈ 0.1 seconds

Time with IVF/HNSW: O(log n) ≈ 0.001 seconds!

SBERT Numerical Example

We have **3 documents** and **1 query** represented as embeddings.
Doc1: "AI and machine learning" → $d_1 = [0.9, 0.7]$
Doc2: "Climate change effects" → $d_2 = [0.2, 0.9]$
Doc3: "Neural networks deep learning" → $d_3 = [0.8, 0.6]$

Query: "artificial intelligence" → $q = [0.85, 0.65]$

QUESTION: **Find the most similar document using cosine similarity.**

SBERT Numerical Example

Step 1: Compute Vector Norms

Query norm

$$\| q \| = \sqrt{0.85^2 + 0.65^2}$$
$$= \sqrt{0.7225 + 0.4225}$$
$$= \sqrt{1.145}$$
$$\approx 1.070$$

Document norms

Doc1

$$\| d_1 \| = \sqrt{0.9^2 + 0.7^2}$$
$$= \sqrt{0.81 + 0.49}$$
$$= \sqrt{1.30}$$
$$\approx 1.140$$

Doc2

$$\| d_2 \| = \sqrt{0.2^2 + 0.9^2}$$
$$= \sqrt{0.04 + 0.81}$$
$$= \sqrt{0.85}$$
$$\approx 0.922$$

Doc3

$$\| d_3 \| = \sqrt{0.8^2 + 0.6^2}$$
$$= \sqrt{0.64 + 0.36}$$
$$= \sqrt{1.00}$$
$$= 1.00$$

SBERT Numerical Example

Step 2: Cosine Similarities

Doc1 similarity

$$cos(q, d_1) = \frac{(0.85 \times 0.9 + 0.65 \times 0.7)}{(1.070 \times 1.140)} = \frac{(0.765 + 0.455)}{1.220} = \frac{1.22}{1.220} \approx 0.999$$

Doc2 similarity

$$cos(q, d_2) = \frac{(0.85 \times 0.2 + 0.65 \times 0.9)}{(1.070 \times 0.922)} = \frac{(0.17 + 0.585)}{0.986} = \frac{0.755}{0.986} \approx 0.77$$

Doc3 similarity

$$cos(q, d_3) = \frac{(0.85 \times 0.8 + 0.65 \times 0.6)}{(1.070 \times 1.00)} = \frac{(0.68 + 0.39)}{1.070} = \frac{1.07}{1.07} \approx 1.00$$

Step 3: Ranking Results

Document	Cosine Similarity
Doc3: Neural networks deep learning	**1.00**
Doc1: AI and machine learning	**0.999**
Doc2: Climate change effects	**0.77**

Popular SBERT Models

Model	Dimensions	Speed	Performance	Use Case
all-MiniLM-L6-v2	384	Very Fast	Good	General-purpose semantic search
all-mpnet-base-v2	768	Slower	Best	High-accuracy embeddings
multi-qa-mpnet-base-dot-v1	768	Slow	Excellent	Question-answer retrieval
paraphrase-MiniLM-L6-v2	384	Fast	Good	Paraphrase detection
msmarco-bert-base-dot-v5	768	Slow	Excellent	Search / document retrieval

Task: Semantic Textual Similarity (STS)

Model	Spearman Correlation	Speed (sentences/sec)
all-MiniLM-L6-v2	84.5	15,000
all-mpnet-base-v2	86.9	3,000

SBERT - Strengths and Limitations

Semantic understanding - catches synonyms and concepts
Fixed-size embeddings - efficient storage and comparison
Fast inference - O(1) per query after indexing
Pre-trained models - works out-of-the-box
Multilingual support - models for 50+ languages

- **Limitations:**
  Loss of token-level interaction - "cat sat on mat" vs "mat sat on cat" have same embedding
  Training data bias - inherits biases from training data
  Out-of-vocabulary - struggles with rare/domain-specific terms
  Context window - limited to 256-512 tokens
  Not interpretable - can't explain why documents matched

Analogy: SBERT is like having a **smart assistant who writes summaries**. The summaries capture the gist, but lose exact wording and word order nuances.

COLBERT AND LATE INTERACTION

What is ColBERT?

Definition

ColBERT (Contextualized Late Interaction over BERT) is a neural retrieval model that combines the **efficiency of bi-encoders** with the **precision of cross-encoders**.

It achieves this by:

- Keeping **token-level embeddings**
- Using a **late interaction scoring mechanism**
- Computing similarity **after encoding**, instead of during encoding.

ColBERT is built on top of **BERT**.

COLBERT AND LATE INTERACTION

What is ColBERT? The Motivation

Cross-Encoder
High accuracy but extremely slow

```
[CLS] query [SEP] document [SEP] → relevance score
```

Problem:

- Must process **every query-document pair**
- Very expensive for large search systems.

Bi-Encoder
Fast but loses fine-grained interaction

```
query → single vector
document → single vector
cosine similarity → score
```

Problem:

- Compresses the entire sentence into **one vector**
- Loses **which words matched which words**.

COLBERT AND LATE INTERACTION

What is ColBERT?

ColBERT Approach (Late Interaction (Best of both)**)**
Instead of one vector, ColBERT keeps **token embeddings**.

$$\text{Query} \rightarrow [q_1, q_2, q_3, \ldots]$$
$$\text{Document} \rightarrow [d_1, d_2, d_3, \ldots]$$

- Similarity is computed between **all query tokens and document tokens**.
- Then ColBERT selects the **best match for each query token**.

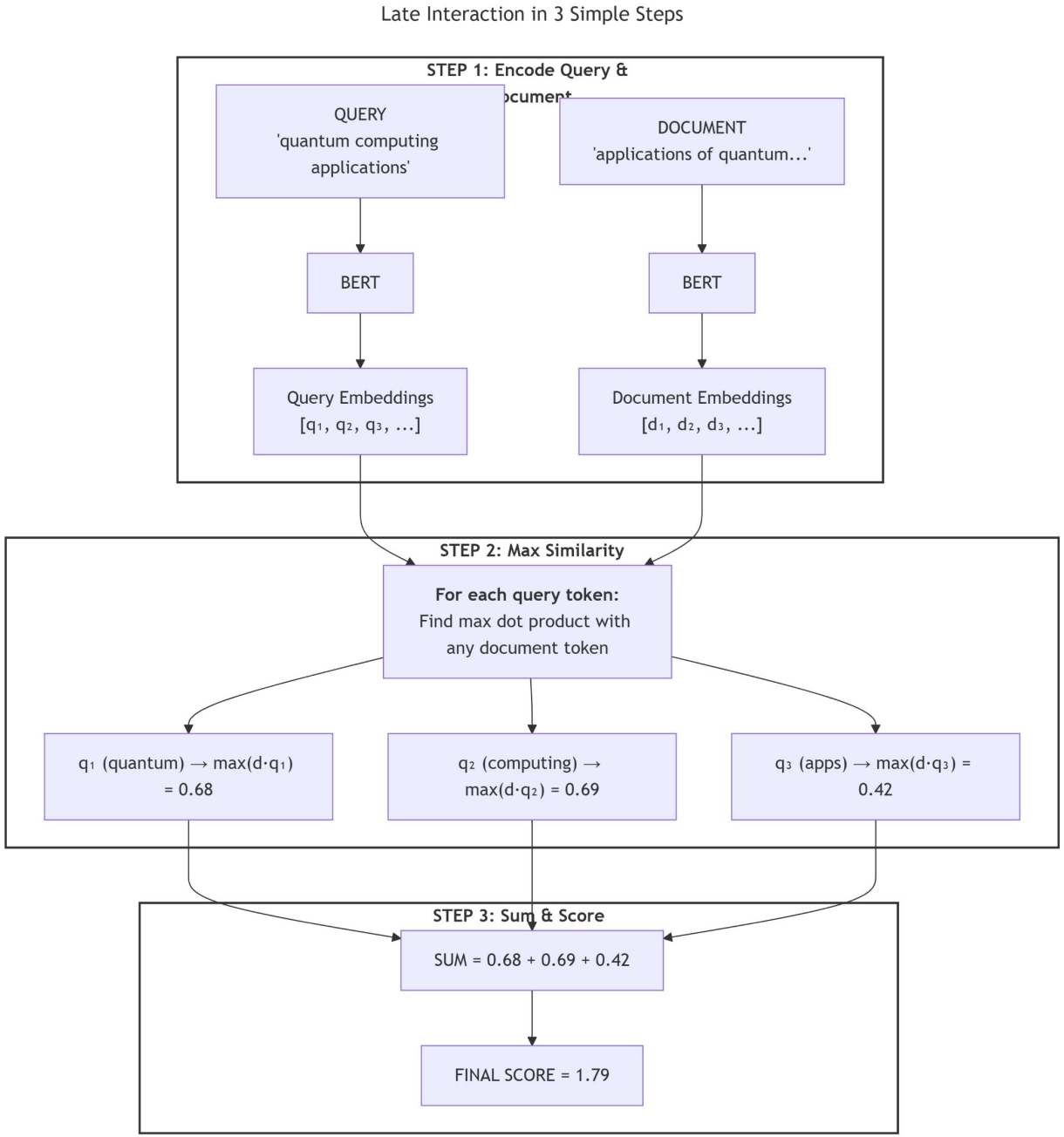
Conceptually:

$$\text{Score(query, document)} = \Sigma \max \text{cosine}(q_{\square}, d_{\square})$$

This preserves **fine-grained word matching** while still allowing **fast retrieval**.

COLBERT AND LATE INTERACTION ARCHITECTURE

What is ColBERT ARCHITECTURE?



ColBERT Scoring – The Math

In **ColBERT**, we compare **each query token with every document token**.

Let:

$Q = [q_1, q_2, ..., q_n]$ → query token embeddings

$D = [d_1, d_2, ..., d_m]$ → document token embeddings

For each query token:

$$\text{max_sim_i} = \max_j \text{cosine}(q_i, d_j)$$

Final document score:

$$\text{Score}(Q,D) = \Sigma \text{ max_sim_i}$$

This is called **MaxSim (maximum similarity)**.

ColBERT Scoring – Numerical Example

Query: **"quantum computing"**
Document: **"quantum physics applications"**
Token embeddings
q_quantum = [0.9, 0.3]
q_computing = [0.2, 0.8]

d_quantum = [0.85, 0.25]
d_physics = [0.1, 0.9]
d_applications= [0.3, 0.6]

ColBERT Scoring – Numerical Example

Step 1: Norms

$$\|q_quantum\| = \sqrt{(0.9^2 + 0.3^2)} = \sqrt{(0.81 + 0.09)} = \sqrt{0.90} = 0.949$$

$$\|q_computing\| = \sqrt{(0.2^2 + 0.8^2)} = \sqrt{(0.04 + 0.64)} = \sqrt{0.68} = 0.825$$

Document tokens:

$$\|d_quantum\| = \sqrt{(0.85^2 + 0.25^2)} = \sqrt{(0.7225 + 0.0625)} = \sqrt{0.785} = 0.886$$

$$\|d_physics\| = \sqrt{(0.1^2 + 0.9^2)} = \sqrt{0.82} = 0.905$$

$$\|d_applications\| = \sqrt{(0.3^2 + 0.6^2)} = \sqrt{0.45} = 0.671$$

ColBERT Scoring – Numerical Example

Step 2: MaxSim for q_quantum

Similarity with "quantum"

(0.9×0.85 + 0.3×0.25) / (0.949 × 0.886)

= (0.765 + 0.075) / 0.841
= 0.84 / 0.841
≈ 0.999

Similarity with "physics"

(0.9×0.1 + 0.3×0.9) / (0.949 × 0.905)

= (0.09 + 0.27) / 0.859
= 0.36 / 0.859
≈ 0.419

Similarity with "applications"

(0.9×0.3 + 0.3×0.6) / (0.949 × 0.671)

= (0.27 + 0.18) / 0.637
= 0.45 / 0.637
≈ 0.706

$max_sim_1 = 0.999$

ColBERT Scoring – Numerical Example

Step 3: MaxSim for q_computing

With "quantum"

(0.2×0.85 + 0.8×0.25) / (0.825 × 0.886)

= (0.17 + 0.20) / 0.731
= 0.37 / 0.731
≈ 0.506

With "physics"

(0.2×0.1 + 0.8×0.9) / (0.825 × 0.905)
= (0.02 + 0.72) / 0.747
= 0.74 / 0.747
≈ 0.991

With "applications"

(0.2×0.3 + 0.8×0.6) / (0.825 × 0.671)
= (0.06 + 0.48) / 0.554
= 0.54 / 0.554
≈ 0.975

max_sim_2 = 0.991

ColBERT Scoring – Numerical Example

Step 4: Final ColBERT Score

$$\text{Score}(Q,D) = \text{max_sim}_1 + \text{max_sim}_2$$
$$= 0.999 + 0.991$$
$$= 1.99$$

- ✔ Unlike SBERT, which compares whole sentence embeddings, ColBERT compares individual tokens and keeps the best match for each query token.
- ✔ This preserves fine-grained word interactions while remaining much faster than cross-encoders.

ColBERT vs SBERT vs Cross-Encoder

This comparison shows three major retrieval architectures built on **BERT**.

- **Cross-encoder** → highest accuracy but slow
- **SBERT** → fastest retrieval
- **ColBERT** → balance between speed and accuracy

Aspect	Cross-Encoder	SBERT	ColBERT
Scoring	Full attention over query + document	Pooled sentence vectors	Token-level MaxSim
Pre-computation	No	Yes (documents)	Yes (documents)
Query-time compute	O(n × BERT) forward passes	O(n × d) cosine similarity	O(m × n × d) token similarity
Accuracy	Very High	Medium	High
Speed	Very Slow	Very Fast	Moderate
Interpretability	High	Low	Medium
Storage per doc	None	~384–768 floats	~token_count × 128/768

ColBERT Problem

Problem
Rank documents for query:
"machine learning"

Query tokens:
["machine", "learning"]
Document 1
"deep learning algorithms"
→ ["deep", "learning", "algorithms"]
Document 2
"coffee machine repair"
→ ["coffee", "machine", "repair"]

Token Embeddings (2D simplified)
q_machine = [0.9, 0.1]
q_learning = [0.2, 0.8]

d1_deep = [0.8, 0.3]
d1_learning = [0.3, 0.7]
d1_algorithms = [0.4, 0.5]

d2_coffee = [0.1, 0.2]
d2_machine = [0.8, 0.2]
d2_repair = [0.1, 0.9]

ColBERT PROBLEM

Step 1: Vector Norms

$||q_machine|| = \sqrt{(0.9^2 + 0.1^2)} = \sqrt{0.82} = 0.906$

$||q_learning|| = \sqrt{(0.2^2 + 0.8^2)} = \sqrt{0.68} = 0.825$

$||d1_deep|| = \sqrt{(0.8^2 + 0.3^2)} = \sqrt{0.73} = 0.854$

$||d1_learning|| = \sqrt{(0.3^2 + 0.7^2)} = \sqrt{0.58} = 0.762$

$||d1_algorithms|| = \sqrt{(0.4^2 + 0.5^2)} = \sqrt{0.41} = 0.640$

$||d2_coffee|| = \sqrt{(0.1^2 + 0.2^2)} = \sqrt{0.05} = 0.224$

$||d2_machine|| = \sqrt{(0.8^2 + 0.2^2)} = \sqrt{0.68} = 0.825$

$||d2_repair|| = \sqrt{(0.1^2 + 0.9^2)} = \sqrt{0.82} = 0.906$

ColBERT PROBLEM

Step 2: Score Document 1
For q_machine

sim(q_machine, d1_deep)

$= (0.9\times0.8 + 0.1\times0.3) / (0.906 \times 0.854)$
$= (0.72 + 0.03) / 0.774$
$= 0.75 / 0.774$
≈ 0.969

sim(q_machine, d1_learning)

$= (0.9\times0.3 + 0.1\times0.7) / (0.906 \times 0.762)$
$= (0.27 + 0.07) / 0.690$
$= 0.34 / 0.690$
≈ 0.493

sim(q_machine, d1_algorithms)

$= (0.9\times0.4 + 0.1\times0.5) / (0.906 \times 0.640)$
$= (0.36 + 0.05) / 0.580$
$= 0.41 / 0.580$
≈ 0.707

$max_sim_1 = 0.969$

ColBERT PROBLEM

Step 2: Score Document 1
For q_learning

sim(q_learning, d1_deep)

$= (0.2\times0.8 + 0.8\times0.3) / (0.825 \times 0.854)$
$= (0.16 + 0.24) / 0.704$
$= 0.40 / 0.704$
≈ 0.568

sim(q_learning, d1_learning)
$= (0.2\times0.3 + 0.8\times0.7) / (0.825 \times 0.762)$
$= (0.06 + 0.56) / 0.628$
$= 0.62 / 0.628$
≈ 0.987

sim(q_learning, d1_algorithms)
$= (0.2\times0.4 + 0.8\times0.5) / (0.825 \times 0.640)$
$= (0.08 + 0.40) / 0.528$
$= 0.48 / 0.528$
≈ 0.909

$max_sim_2 = 0.987$

Doc1 Score = 0.969 + 0.987 = 1.956

ColBERT PROBLEM

Step 3: Score Document 2
For q_machine

sim(q_machine, d2_coffee)
= (0.9×0.1 + 0.1×0.2) / (0.906 × 0.224)
= (0.09 + 0.02) / 0.203
= 0.11 / 0.203
≈ 0.542

sim(q_machine, d2_machine)
= (0.9×0.8 + 0.1×0.2) / (0.906 × 0.825)
= (0.72 + 0.02) / 0.748
= 0.74 / 0.748
≈ 0.989

sim(q_machine, d2_repair)
= (0.9×0.1 + 0.1×0.9) / (0.906 × 0.906)
= (0.09 + 0.09) / 0.821
= 0.18 / 0.821
≈ 0.219

max_sim_1 = 0.989

ColBERT PROBLEM

Step 3: Score Document 2
For q_learning

sim(q_learning, d2_coffee)
= (0.2×0.1 + 0.8×0.2) / (0.825 × 0.224)
= (0.02 + 0.16) / 0.185
= 0.18 / 0.185
≈ 0.973

sim(q_learning, d2_machine)
= (0.2×0.8 + 0.8×0.2) / (0.825 × 0.825)
= (0.16 + 0.16) / 0.681
= 0.32 / 0.681
≈ 0.470

sim(q_learning, d2_repair)
= (0.2×0.1 + 0.8×0.9) / (0.825 × 0.906)
= (0.02 + 0.72) / 0.748
= 0.74 / 0.748
≈ 0.989

max_sim_2 = 0.989

Doc2 Score = 0.989 + 0.989 = 1.978

ColBERT PROBLEM FINAL RANKING:

Document	Score
Document 2: "coffee machine repair"	**1.978**
Document 1: "deep learning algorithms"	**1.956**

This example reveals a known behavior of ColBERT:
The model may match query tokens with semantically similar but contextually different document tokens.

Example:

```
learning ↔ repair
```

This happens because MaxSim selects the best token match independently, without enforcing global context alignment.

ColBERT Strengths vs Weaknesses :

Strengths	Weaknesses
Preserves word-level interactions (knows which words matched)	Higher storage requirements (stores token embeddings)
Better than SBERT for fine-grained relevance	Slower than SBERT due to O(m × n) token comparisons
Faster than cross-encoders because document embeddings are pre-computed	May match unrelated words with similar embeddings
Provides interpretability by showing token-level matches	Limited positional understanding between tokens
Handles multi-word expressions effectively	Implementation complexity is higher than SBERT

Storage Comparison:

Model	Storage Formula	Storage for 1M Documents
SBERT	1M × 768 × 4 bytes	~3.07 GB
ColBERT (raw)	1M × 100 × 768 × 4 bytes	~307 GB
ColBERT (with compression)	Residual compression + binarization	~30–40 GB

ColBERTv2 - Improvements:

Technique	Idea	Benefit
Residual Compression	Store token embeddings as centroid ID + residual difference	Reduces storage drastically
Denoised Supervision	Use cross-encoder predictions as training labels (knowledge distillation)	Improves training quality
Centroid Interaction	Compare cluster centroids before full token comparison	Faster retrieval

ColBERTv2 - Improvements:

Residual Compression: This reduces storage while keeping most of the semantic information.

Step	Description
Original embedding	[0.234, -0.567, 0.891, ...]
Compression	Store centroid ID + small residual
Storage reduction	32-bit float → 2–4 bits per value

Denoised Supervision:

Step	**Description**
Teacher model	Cross-encoder generates relevance scores
Student model	ColBERT learns from these scores
Result	Cleaner training data and better ranking

ColBERTv2 - Improvements:

Centroid Interaction : This significantly reduces computation..

Step	Description
Token clustering	Document tokens grouped into clusters
Fast filtering	Compare query tokens with cluster centroids first
Final scoring	Full comparison only on top clusters

Performance Improvements

Metric	Original ColBERT	ColBERTv2
Storage	~307 GB	~16 GB
Speed	Baseline	~2× faster
Accuracy	Strong	Higher than ColBERT

HYBRID RETRIEVAL :

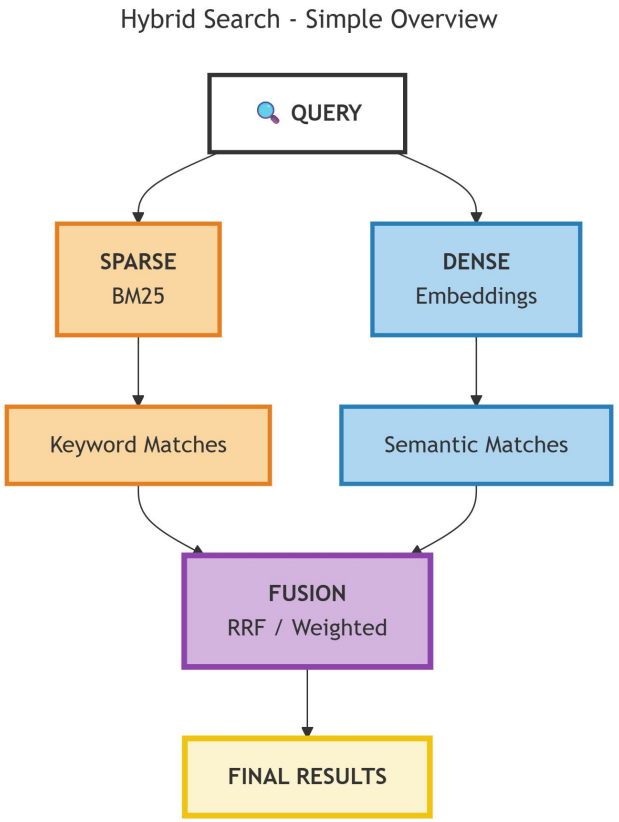
What is Hybrid Retrieval?

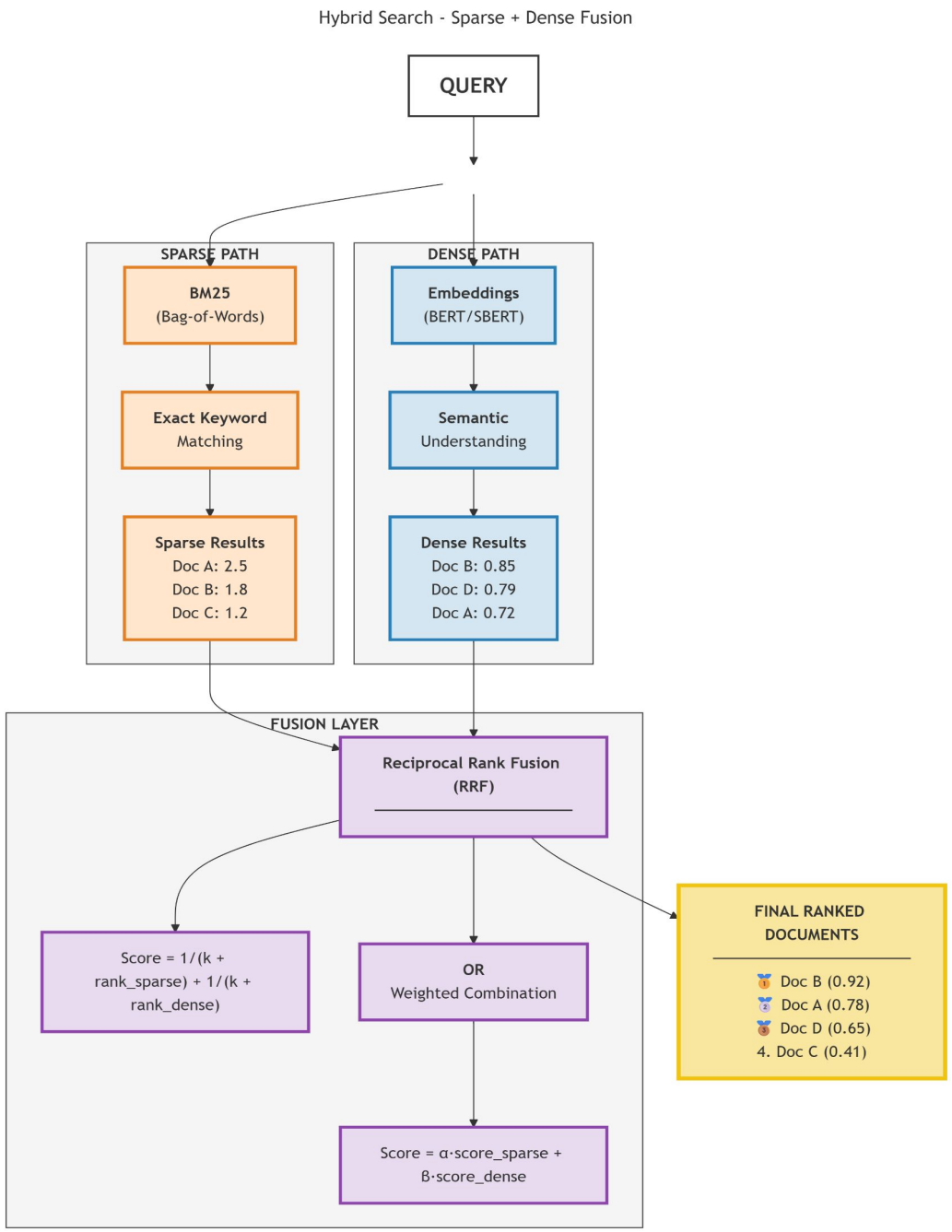
Hybrid Retrieval combines sparse retrieval (BM25) and dense retrieval (SBERT/ColBERT) to leverage the strengths of both approaches while mitigating their weaknesses.

Step	Sparse Path (BM25)	Dense Path (Embeddings)
Input	Query	Query
Retrieval	Exact keyword matching	Semantic understanding via embeddings
Output	Ranked documents	Ranked documents
Fusion	Combined using RRF (Reciprocal Rank Fusion) or weighted scores	Combined using RRF or weighted scores

Analogy: Hybrid retrieval is like having **two expert researchers**:
One finds **exact quotes** (BM25)
One understands **concepts and related ideas** (Dense)
Then you **combine their recommendations** for the best results.

HYBRID RETRIEVAL :



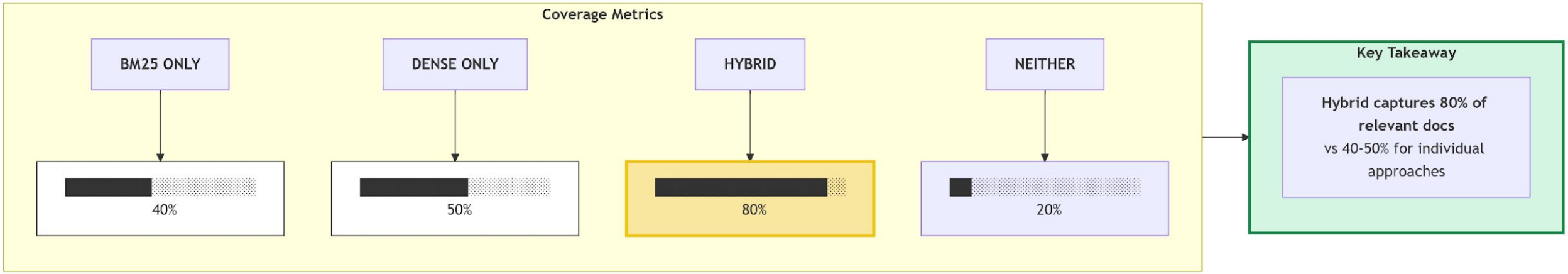


HYBRID RETRIEVAL :

When Each Method Excels

Scenario	Example	BM25	Dense
Exact match needed	"iPhone 15 Pro Max price"	Perfect	May return related products
Synonyms	"automobile" vs "car"	Misses	Catches
Rare terms	"qubit superposition"	Good	May not understand
Ambiguous terms	"apple" (fruit vs company)	Confused	Context-aware
Long-tail queries	"1920s fashion trends"	Good	May hallucinate
Cross-lingual	"chat" (English/French)	Fails	Works

Coverage Comparison



HYBRID RETRIEVAL : Fusion Methods – How to Combine Sparse & Dense Scores

Method 1: Reciprocal Rank Fusion (RRF)
Formula:

$$\text{score}(d) = \sum_{s} \frac{1}{k + rank_s(d)}$$

Where:
•$\text{rank}_s(d)$ =rank of document $d$in system s
•k= constant (typically 60)

Document	BM25 Rank	Dense Rank	RRF Calculation	RRF Score
A	3	5	1/(60+3) + 1/(60+5)	0.0159 + 0.0154 = 0.0313
B	10	2	1/(60+10) + 1/(60+2)	0.0143 + 0.0161 = 0.0304

Document A wins because it ranks well in both systems, even if Dense rank is slightly lower.

HYBRID RETRIEVAL :

Method 2: Weighted Sum

$$\mathbf{score}(\boldsymbol{d}) = \boldsymbol{\alpha} \times \boldsymbol{norm_score_bm25}(\boldsymbol{d}) + (\mathbf{1} - \boldsymbol{\alpha}) \times \boldsymbol{norm_score_dense}(\boldsymbol{d})$$

- α typically 0.3–0.5
- Requires normalized scores from each system

Method 3: Score Normalization (pre-weighting)

- Min-Max: $\text{norm_}score = \frac{score - min}{max - min}$
- Z-score: $\text{norm_}score = \frac{score - mean}{std_dev}$

HYBRID RETRIEVAL : RRF Numerical Example (k=60)

Doc	BM25 Rank	Dense Rank	RRF Calculation	RRF Score
D1	1	3	1/(60+1) + 1/(60+3)	0.0164 + 0.0159 = 0.0323
D2	2	5	1/(60+2) + 1/(60+5)	0.0161 + 0.0154 = 0.0315
D3	4	1	1/(60+4) + 1/(60+1)	0.0156 + 0.0164 = 0.0320
D4	3	8	1/(60+3) + 1/(60+8)	0.0159 + 0.0147 = 0.0306
D5	10	2	1/(60+10) + 1/(60+2)	0.0143 + 0.0161 = 0.0304

Ranking by RRF:

D1 (0.0323) → D3 (0.0320) → D2 (0.0315) → D4 (0.0306) → D5 (0.0304)

Key Insight: Documents performing well in both sparse and dense systems get the **highest RRF scores**.

HYBRID RETRIEVAL :

Problem

Find documents relevant to **“machine learning algorithms”** using **BM25 + Dense (SBERT)**.

HYBRID RETRIEVAL :

Step 1: BM25 Scores (Sparse)

Document	BM25 Score
Doc 1: "machine learning algorithms for beginners"	12.5
Doc 2: "coffee machine cleaning instructions"	3.2
Doc 3: "deep learning neural networks tutorial"	8.7
Doc 4: "algorithm design and analysis"	6.1
Doc 5: "machine learning in healthcare"	10.3

Step 2: Dense Scores (SBERT)

Query embedding: [0.7, 0.5, 0.3, ...]

Document 1 embedding: [0.8, 0.4, 0.2, ...] → cosine = 0.95
Document 2 embedding: [0.1, 0.2, 0.9, ...] → cosine = 0.45
Document 3 embedding: [0.6, 0.7, 0.3, ...] → cosine = 0.92
Document 4 embedding: [0.3, 0.2, 0.8, ...] → cosine = 0.56
Document 5 embedding: [0.7, 0.6, 0.2, ...] → cosine = 0.98

Document	Cosine Similarity
Doc 1	0.95
Doc 2	0.45
Doc 3	0.92
Doc 4	0.56
Doc 5	0.98

HYBRID RETRIEVAL :

Step 3: Normalize Scores (Min-Max)

$$\text{norm_}score = \frac{score - min}{max - min}$$

BM25 min = 3.2, max = 12.5
Dense min = 0.45, max = 0.98

Document 1: BM25_norm = (12.5-3.2)/(12.5-3.2) = 1.00
Dense_norm = (0.95-0.45)/(0.98-0.45) = 0.50/0.53 = 0.94
Document 2: BM25_norm = (3.2-3.2)/(12.5-3.2) = 0.00
Dense_norm = (0.45-0.45)/(0.98-0.45) = 0.00
Document 3: BM25_norm = (8.7-3.2)/(12.5-3.2) = 5.5/9.3 = 0.59
Dense_norm = (0.92-0.45)/(0.98-0.45) = 0.47/0.53 = 0.89
Document 4: BM25_norm = (6.1-3.2)/(12.5-3.2) = 2.9/9.3 = 0.31
Dense_norm = (0.56-0.45)/(0.98-0.45) = 0.11/0.53 = 0.21
Document 5: BM25_norm = (10.3-3.2)/(12.5-3.2) = 7.1/9.3 = 0.76
Dense_norm = (0.98-0.45)/(0.98-0.45) = 1.00

HYBRID RETRIEVAL :

Step 4: Weighted Combination (α = 0.4 for BM25, 0.6 for Dense)

$$\text{Hybrid } Score = 0.4 \times BM25_norm + 0.6 \times Dense_norm$$

Document 1: 0.4 × 1.00 + 0.6 × 0.94 = 0.40 + 0.564 = 0.964
Document 2: 0.4 × 0.00 + 0.6 × 0.00 = 0.000
Document 3: 0.4 × 0.59 + 0.6 × 0.89 = 0.236 + 0.534 = 0.770
Document 4: 0.4 × 0.31 + 0.6 × 0.21 = 0.124 + 0.126 = 0.250
Document 5: 0.4 × 0.76 + 0.6 × 1.00 = 0.304 + 0.600 = 0.904

Document	Hybrid Score
Doc 1	0.964
Doc 2	0.000
Doc 3	0.770
Doc 4	0.250
Doc 5	0.904

HYBRID RETRIEVAL :

Step 5: Final Ranking

Rank	Document	Hybrid Score	BM25 Only Rank	Dense Only Rank
1	Doc 1	0.964	1	3
2	Doc 5	0.904	2	1
3	Doc 3	0.770	3	2
4	Doc 4	0.250	4	4
5	Doc 2	0.000	5	5

Key Observations:

- Doc 1 wins because it's strong in both
- Doc 5 (strong dense, good BM25) comes second
- Doc 3 (strong dense, moderate BM25) comes third
- Doc 4 (weak in both) is appropriately low

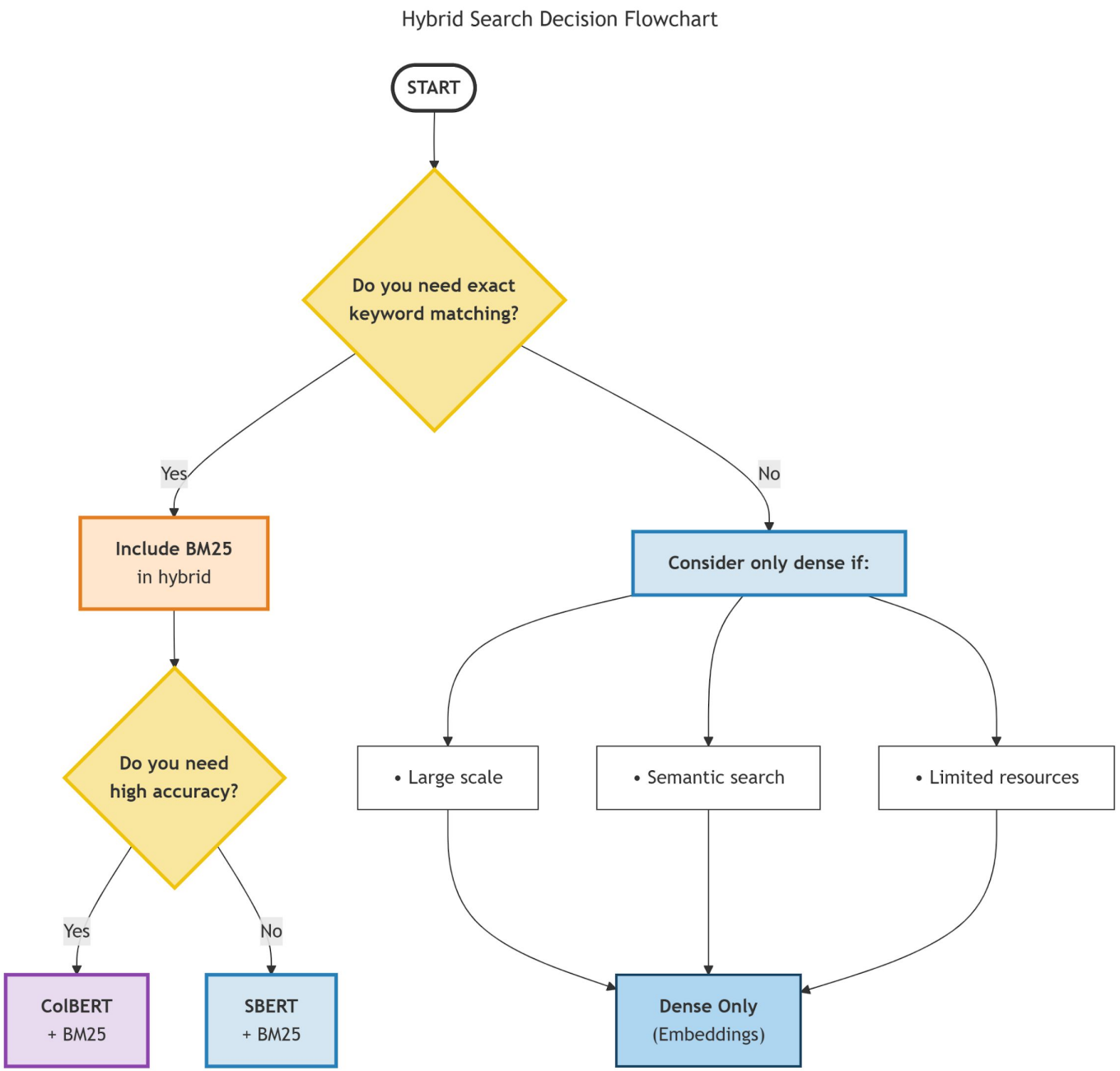
HYBRID RETRIEVAL : Performance Metrics - Hybrid vs Individual

Benchmark Results – MS MARCO Dataset

Method	MRR@10	Recall@100	Latency (ms)
BM25 Only	0.184	0.753	2
Dense Only (SBERT)	0.331	0.887	10
Dense Only (ColBERT)	0.387	0.912	25
Hybrid (BM25+SBERT)	0.358	0.921	12
Hybrid (BM25+ColBERT)	0.401	0.938	27
Hybrid + RRF	0.412	0.945	28

GenerativeAI and Its Applications

WHEN WHICH APPROACH?



Problem 1: SBERT Similarity Calculation

Given these 2D embeddings (simplified):

- ✔ Query: "artificial intelligence" → [0.6, 0.8]
- ✔ Doc1: "machine learning" → [0.7, 0.6]
- ✔ Doc2: "climate change" → [0.2, 0.3]
- ✔ Doc3: "deep learning" → [0.8, 0.5]

Calculate cosine similarity for each document and rank them.

Problem 2: ColBERT Scoring

Query tokens: ["neural", "networks"]
Document tokens: ["deep", "learning", "neural", "networks"]

Given embeddings:

- q_neural = [0.9, 0.2]
- q_networks = [0.3, 0.8]
- d_deep = [0.8, 0.3]
- d_learning = [0.2, 0.7]
- d_neural = [0.85, 0.25]
- d_networks = [0.35, 0.75]

Calculate ColBERT score.

Problem 3: Hybrid Fusion

BM25 ranks: Doc1(2), Doc2(5), Doc3(1), Doc4(3)
Dense ranks: Doc1(3), Doc2(1), Doc3(4), Doc4(2)

Calculate RRF scores with k=60 and determine final ranking.

THANK YOU